

Schema Design

```
-- Database: SQLite 3
-- Schema: Normalized relational design

CREATE TABLE users (
    user_id      TEXT PRIMARY KEY,
    location     TEXT NOT NULL,
    language     TEXT NOT NULL,
    account_created TEXT NOT NULL,
    follower_count INTEGER NOT NULL
);

CREATE TABLE posts (
    post_id      TEXT PRIMARY KEY,
    user_id      TEXT NOT NULL,
    platform     TEXT,
    text_content TEXT,
    timestamp    TEXT NOT NULL,
    likes        REAL,
    shares       INTEGER NOT NULL,
    comments     INTEGER NOT NULL,
    FOREIGN KEY (user_id) REFERENCES users(user_id)
);

CREATE TABLE posts_corrupted (
    post_id TEXT PRIMARY KEY, user_id TEXT,
    platform TEXT, text_content TEXT,
    timestamp TEXT, likes TEXT,
    shares TEXT, comments TEXT
);

-- Performance Indexes
CREATE INDEX idx_posts_user_id ON posts(user_id);
CREATE INDEX idx_posts_platform ON posts(platform);
CREATE INDEX idx_users_follower_count ON users(follower_count);
```

E1 - Platform Popularity

```
SELECT platform,  
       COUNT(*) AS num_posts  
FROM posts  
WHERE platform IS NOT NULL  
GROUP BY platform  
ORDER BY num_posts DESC;
```

E2 - Most Engaged Posts (Top 10)

```
SELECT post_id,  
       platform,  
       likes,  
       shares,  
       comments,  
       (likes + shares + comments) AS total_engagement  
FROM posts  
WHERE likes IS NOT NULL  
ORDER BY total_engagement DESC  
LIMIT 10;
```

E3 - Average Engagement by Platform

```
SELECT platform,  
       ROUND(AVG(likes), 2) AS avg_likes,  
       ROUND(AVG(shares), 2) AS avg_shares,  
       ROUND(AVG(comments), 2) AS avg_comments,  
       ROUND(AVG(likes) + AVG(shares) + AVG(comments), 2) AS avg_total_engagement  
FROM posts  
WHERE platform IS NOT NULL  
GROUP BY platform  
ORDER BY avg_total_engagement DESC;
```

E4 - Highly Shared but Poorly Liked

```
SELECT post_id,  
       platform,  
       likes,  
       shares,  
       comments  
FROM posts  
WHERE shares > 1500  
      AND likes < 500  
ORDER BY shares DESC;
```

E5 - Users With Large Audiences (>40K followers)

```
SELECT user_id,  
       location,  
       language,  
       follower_count  
FROM users  
WHERE follower_count > 40000  
ORDER BY follower_count DESC;
```

M1 - Which Locations Generate the Most Engagement?

```
SELECT u.location,  
       COUNT(p.post_id) AS num_posts,  
       ROUND(SUM(COALESCE(p.likes, 0) + p.shares + p.comments), 0) AS total_engagement  
FROM posts p  
JOIN users u ON p.user_id = u.user_id  
GROUP BY u.location  
ORDER BY total_engagement DESC;
```

M2 - Do High Follower Users Get More Engagement?

```
SELECT
  CASE
    WHEN u.follower_count >= 25000 THEN 'High (>=25K)'
    ELSE 'Low (<25K)'
  END AS follower_group,
  ROUND(AVG(COALESCE(p.likes, 0) + p.shares + p.comments), 2) AS avg_engagement_per_post
FROM posts p
JOIN users u ON p.user_id = u.user_id
GROUP BY follower_group
ORDER BY avg_engagement_per_post DESC;
```

M3 - Most Active Users (Top 10)

```
SELECT p.user_id,  
       u.follower_count,  
       u.location,  
       COUNT(p.post_id) AS post_count,  
       ROUND(SUM(COALESCE(p.likes, 0) + p.shares + p.comments), 0) AS total_engagement  
FROM posts p  
JOIN users u ON p.user_id = u.user_id  
GROUP BY p.user_id  
ORDER BY post_count DESC, total_engagement DESC  
LIMIT 10;
```

M4 - Platform Behaviour by High Follower Users (>=30K)

```
SELECT p.platform,  
       ROUND(AVG(COALESCE(p.likes, 0) + p.shares + p.comments), 2) AS avg_engagement_per_post  
FROM posts p  
JOIN users u ON p.user_id = u.user_id  
WHERE u.follower_count >= 30000  
      AND p.platform IS NOT NULL  
GROUP BY p.platform  
ORDER BY avg_engagement_per_post DESC;
```

M5 - Detect Suspicious Engagement (Top 20)

```
SELECT post_id,  
       platform,  
       likes,  
       shares,  
       comments,  
       (COALESCE(likes, 0) + shares + comments) AS total_engagement  
FROM posts  
WHERE shares > (COALESCE(likes, 0) + comments)  
ORDER BY total_engagement DESC  
LIMIT 20;
```

H1 - Users With Abnormally High Engagement

```
WITH user_engagement AS (  
    SELECT p.user_id,  
           u.location,  
           u.follower_count,  
           COUNT(p.post_id) AS post_count,  
           ROUND(AVG(COALESCE(p.likes, 0) + p.shares + p.comments), 2) AS avg_engagement  
    FROM posts p  
    JOIN users u ON p.user_id = u.user_id  
    GROUP BY p.user_id  
,  
overall AS (  
    SELECT AVG(COALESCE(likes, 0) + shares + comments) AS overall_avg  
    FROM posts  
)  
SELECT ue.user_id,  
       ue.location,  
       ue.follower_count,  
       ue.post_count,  
       ue.avg_engagement,  
       ROUND(o.overall_avg, 2) AS overall_avg,  
       ROUND(2 * o.overall_avg, 2) AS threshold  
FROM user_engagement ue, overall o  
WHERE ue.avg_engagement > 2 * o.overall_avg  
ORDER BY ue.avg_engagement DESC;
```

H2 - Rank Users Within Their Location (Top 3)

```
WITH user_location_engagement AS (  
    SELECT p.user_id,  
           u.location,  
           SUM(COALESCE(p.likes, 0) + p.shares + p.comments) AS total_engagement,  
           RANK() OVER (  
               PARTITION BY u.location  
               ORDER BY SUM(COALESCE(p.likes, 0) + p.shares + p.comments) DESC  
           ) AS location_rank  
    FROM posts p  
    JOIN users u ON p.user_id = u.user_id  
    GROUP BY p.user_id, u.location  
)  
SELECT user_id,  
       location,  
       ROUND(total_engagement, 0) AS total_engagement,  
       location_rank  
FROM user_location_engagement  
WHERE location_rank <= 3  
ORDER BY location, location_rank;
```

H3 - Exceptional Posts ($\geq 2\times$ Platform Average)

```
WITH platform_avg AS (  
    SELECT platform,  
           AVG(COALESCE(likes, 0) + shares + comments) AS avg_engagement  
    FROM posts  
    WHERE platform IS NOT NULL  
    GROUP BY platform  
)  
SELECT p.post_id,  
       p.platform,  
       ROUND(COALESCE(p.likes, 0) + p.shares + p.comments, 0) AS post_engagement,  
       ROUND(pa.avg_engagement, 2) AS platform_avg_engagement  
FROM posts p  
JOIN platform_avg pa ON p.platform = pa.platform  
WHERE (COALESCE(p.likes, 0) + p.shares + p.comments) >= 2 * pa.avg_engagement  
ORDER BY post_engagement DESC  
LIMIT 20;
```

H4 - Follower to Engagement Anomaly

```
WITH user_totals AS (  
    SELECT p.user_id,  
           u.follower_count,  
           u.location,  
           SUM(COALESCE(p.likes, 0) + p.shares + p.comments) AS total_engagement,  
           COUNT(p.post_id) AS post_count  
    FROM posts p  
    JOIN users u ON p.user_id = u.user_id  
    GROUP BY p.user_id  
,  
percentile_threshold AS (  
    SELECT total_engagement AS threshold  
    FROM user_totals  
    ORDER BY total_engagement DESC  
    LIMIT 1 OFFSET (SELECT CAST(COUNT(*) * 0.10 AS INTEGER) FROM user_totals)  
)  
SELECT ut.user_id,  
       ut.follower_count,  
       ut.location,  
       ut.post_count,  
       ROUND(ut.total_engagement, 0) AS total_engagement  
FROM user_totals ut, percentile_threshold pt  
WHERE ut.follower_count < 5000  
      AND ut.total_engagement >= pt.threshold  
ORDER BY ut.total_engagement DESC;
```

H5 - Identify Data Anomalies (Corrupted Dataset)

```

SELECT post_id,
       CASE
         WHEN (CAST(likes AS REAL) < 0) AND (platform IS NULL OR platform = '')
              AND (text_content IS NULL OR text_content = '')
              AND (text_content LIKE '%&%' OR text_content LIKE '%<div>%' OR text_content LIKE '%<br>')
         THEN 'Negative likes | Missing platform | Missing text | HTML tags'
         WHEN (CAST(likes AS REAL) < 0) AND (platform IS NULL OR platform = '')
         THEN 'Negative likes | Missing platform'
         WHEN (CAST(likes AS REAL) < 0) AND (text_content LIKE '%&%' OR text_content LIKE '%<div>%' OR text_content LIKE '%<br>')
         THEN 'Negative likes | HTML tags'
         WHEN (platform IS NULL OR platform = '') AND (text_content IS NULL OR text_content = '')
         THEN 'Missing platform | Missing text'
         WHEN CAST(likes AS REAL) < 0
         THEN 'Negative likes'
         WHEN platform IS NULL OR platform = ''
         THEN 'Missing platform'
         WHEN text_content IS NULL OR text_content = ''
         THEN 'Missing text content'
         WHEN text_content LIKE '%&%' OR text_content LIKE '%<div>%'
              OR text_content LIKE '%<br>%' OR text_content LIKE '%</div>%'
         THEN 'Contains HTML tags/entities'
         ELSE NULL
       END AS anomaly_type
FROM posts_corrupted
WHERE (CAST(likes AS REAL) < 0)
      OR (platform IS NULL OR platform = '')
      OR (text_content IS NULL OR text_content = '')
      OR (text_content LIKE '%&%' OR text_content LIKE '%<div>%'
          OR text_content LIKE '%<br>%' OR text_content LIKE '%</div>%')
ORDER BY anomaly_type;

```

H6 - Most Suspicious High Impact Users

```

WITH user_stats AS (
    SELECT p.user_id,
           u.location,
           u.follower_count,
           COUNT(p.post_id) AS post_count,
           ROUND(AVG(COALESCE(p.likes, 0) + p.shares + p.comments), 2) AS avg_engagement,
           ROUND(SUM(COALESCE(p.likes, 0) + p.shares + p.comments), 0) AS total_engagement,
           MAX(CASE WHEN p.shares > COALESCE(p.likes, 0) THEN 1 ELSE 0 END) AS has_shares_gt_likes
    FROM posts p
    JOIN users u ON p.user_id = u.user_id
    GROUP BY p.user_id
),
overall AS (
    SELECT AVG(COALESCE(likes, 0) + shares + comments) AS overall_avg
    FROM posts
)
SELECT us.user_id,
       us.location,
       us.follower_count,
       us.post_count,
       us.avg_engagement,
       us.total_engagement
FROM user_stats us, overall o
WHERE us.follower_count < 10000
      AND us.avg_engagement > o.overall_avg
      AND us.has_shares_gt_likes = 1
ORDER BY us.total_engagement DESC;

```